

appsec-review

An automated application security engineer: executive design overview

VERSION

0.6

STATUS

Draft: built, first live report pending

DATE

2026-09-29

OWNER

W. Sollers

AUDIENCE Security leadership, engineering leaders and review operators

Template. Amber *TBD* marks are fill-ins; compile with `\usepackage[final]{appsec-house}` to hide this line.

BOTTOM LINE

appsec-review is an automated application security engineer: a pinned, repeatable process that does the routine, evidence-heavy work of an AppSec review (scope, build, scan, threat-model, read code, argue and verify each finding, score, write up) across more code and more targets than one person can. It is *almost* an engineer, never a replacement. People keep the judgement that carries accountability: what the risk means to the business, what to accept, and whether the report goes out. Every lane is built and unit tested; no target has yet run through to a report.

LIFECYCLE JOBS

81

In the job graph, 2026-09-29

QUALIFIED JOBS

10

Of 81, per the generated readiness view

REVIEW STAGES

4

Red, blue, verifier, scorer: separate roles

TARGETS REPORTED

0 of 4

Run-to-report goal, ADR-0013

ENGINEER HOURS PER REVIEW

TBD: first runs

With and without the process

ELAPSED TIME PER REVIEW

TBD: first runs

Staging to draft report

MODEL SPEND PER REVIEW

TBD: first runs

Per target and budget tier

What leadership should know

- **It does the routine work, not the accountability.** Machines gather, build, scan, read and argue; a named person accepts risk and signs the final package.
- **A tool hit is never a finding.** A finding has passed red-team, blue-team and independent verification, and every claim cites evidence that resolves to exact bytes.
- **Silence is never “clean”.** A scanner that did not run or a build that failed is a coverage gap in the report, never “no issues found”.
- **Python keeps the books; the model supplies judgement only** (ADR-0013). Ids, hashes, citations, scores and reachability are computed, never taken from the model.
- **It is not yet proven end to end.** Ten jobs are qualified; all four targets have run part of the flow and 108 breakages are logged, but none has produced its report yet.

What changes for the organisation

- AppSec engineers review verified findings, gaps and attack paths instead of building targets and triaging raw scanner output.
- Every review yields the same package (findings, coverage, gaps, provenance), so reviews compare across teams and over time. Depth is a setting (`probe`, `standard`, `full`), not a staffing decision.
- New standing duties: persona ownership, reference-feed upkeep and sign-off of each final package.

Contents

1	What the process is, and where the line is	2
2	The engineer's job, mapped to the pipeline	3
3	How a review moves through the system	4
4	Why it scales	4
5	Why it can be trusted	5
6	Where people stay in the loop	6
7	Readiness	6
8	Risk and next decisions	7
A	Evidence, search and reference data	8
B	Authority and operation	8
C	Revision history	8

1 What the process is, and where the line is

An application security engineer spends most of a review on work that follows a method: find out what the system is, get it to build, run the tools, read what they say, model the threats, read the risky code, check each suspicion against the evidence, rate it and write it down. appsec-review encodes that method as a graph of 81 lifecycle jobs run by Dagster. Each job gets bounded input and publishes an immutable attempt with status, coverage, permissions and lineage. Downstream jobs re-check hashes rather than trusting copied prose.

The process reviews an authorized snapshot: source, build metadata, binaries, images, IaC, dependencies, configuration and documents. It may compile the target to recover compiler semantics. Build scripts are treated as hostile and run in a disposable, no-network, least-privilege container. It does not probe production, call live cloud APIs, run authenticated DAST, exercise network services or attempt live exploitation.

The line between the process and the engineer is set by what can be checked. Where a conclusion can be derived from evidence (does this path exist, does this call reach that sink, what CVSS score follows from these metrics), Python derives it. Where it needs judgement about code (is this a plausible attack, does this mitigation hold), a model persona gives an opinion and Python validates the form and the citations. Where it needs accountability (is this risk acceptable to the business, may this report be published), a person decides. Section 6 lists those decisions.

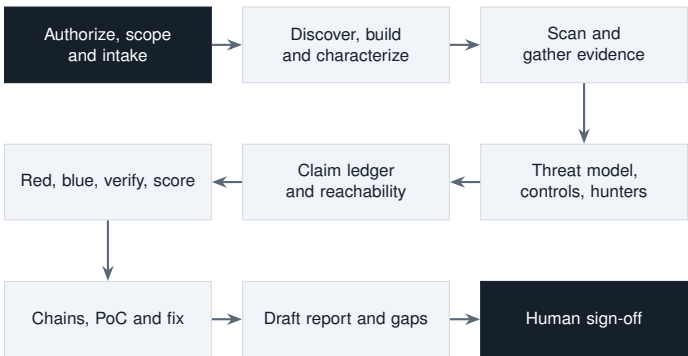
2 The engineer's job, mapped to the pipeline

Each row is part of an AppSec engineer's job. "Python" is deterministic code whose output is re-derived on validation. "Model" is a persona call through the Claude CLI whose reply is checked against a schema and its citations before anything is kept. Job names are those in `job-graph.json`.

ENGINEER TASK	JOBS	PYTHON (DETERMINISTIC)	MODEL (JUDGEMENT)
Scope and characterise the target	00-intake; partition, developer, DevOps and SRE discovery; 01-component-characterization; 02-full-review-input-assembly	Source identity, permissions, scope validation (every file in exactly one scope), derived component and relationship ids, closed review plan, explicit <code>SKIPPED_NA</code> , wave dispatch	Partitions, component purpose, ownership and boundaries
Build it	02-build-index, -classify, -plan, -resolution, -configure; 02-native-build	Unit enumeration, sandboxed offline trials, build locks, compile databases, binary hashes. A unit that will not build is a named gap	Build classification and the per-unit build plan
Run the scanners and read their output	02-source-sast; eight 02-codeql-<lang> nodes; native SAST, IR, code property graph; secrets, IaC, container, SBOM, SCA, licence, lifecycle, binary and mobile jobs; 02-evidence-index, 02-evidence-assembly	Pinned tools in network-disabled containers; leads normalised, mapped to CWE, merged per path and line, tiered P1-P3 (ADR-0015); every accepted lead becomes a candidate claim	Binding producers in evidence assembly. Reading the leads happens in the review stages below
Threat-model	03-threat-model-dfd-stride (workbench), 03-threat-model-reconciliation	DFD, trust boundaries and STRIDE hypotheses from the component map; ids, citations, join and validator replay; reconciliation with the evidence	Workbench cells: privacy (LINDDUN), deployment topology, abuse scenarios, attack trees, supply chain (ADR-0019)
Check controls	04-owasp-validation-worklist, 04-asvs-masvs, 15-stig-srg-validation-worklist, 15-deployment-hardening	Control selection, applicability, tailoring, separate denominators, status matrix; no compliance verdict	OWASP validator cells
Hunt for bugs by reading code	07-hypothesis-discovery (hunter pool)	Shards by component, pins files, resolves every cited location against pinned bytes, drops or defers what does not resolve, deduplicates and tiers (ADR-0018)	General and known-list hunters propose location, weakness class and mechanism
Keep the case file	claim-ledger-routing	Admits candidates from four sources (threat model and OWASP routes, tool leads, hunters, dependency reachability) with ids, citations and proof obligations	None
Make the case, then argue against it	07-red-team-adversarial, 08-blue-team-refutation (sharded reviewer pools)	Shards claims, assigns a distinct persona per shard, merges, applies stage rules, requires one decision per claim (ADR-0021, ADR-0024)	Red team argues exploitability; blue team refutes, narrows or leaves unresolved
Verify independently	09-independent-verification	Only this stage can verify. Scoring and the findings section read verified claims only, whatever their severity	Verifier personas decide whether the cited mechanism is established
Check whether a dependency flaw is reachable	06-reachability-codeql, 06-reachability-ir, 06-cve-reachability	Per-ecosystem engines, a fixed verdict lattice and a hash-bound witness path. Model text never decides reachability (ADR-0022, ADR-0023)	None
Score	12-scoring-prioritization	CVSS 4.0 vector and score by the FIRST algorithm; EPSS/KEV "as of" a dated snapshot; Critical requires a REACHABLE witness, otherwise the cap is High (ADR-0020)	Scorer personas give the eleven base metrics with a reason each, and the CWE
Chain findings into attack paths	14-attack-chain-composition, 14-attack-chain-refutation (lane 14)	Seeds, edge basis (code fact, model flow or synthetic), chain state set by its weakest link; ceiling supported, never a verified finding (ADR-0016)	Composer proposes chains; refuter attacks the weakest link

ENGINEER TASK	JOBS	PYTHON (DETERMINISTIC)	MODEL (JUDGEMENT)
Propose a fix	11-remediation-proposal, 12b-poc-and-fix (lane 12b)	Selects verified, Critical, REACHABLE findings; checks citations and diff scope; a denylist withholds unsafe text. Nothing is executed, compiled or applied; fixes are labelled PATCH_PROPOSED_UNVALIDATED	Light static proof of concept and a proposed fix as a diff
Write the report	10-synthesis-report, completeness-audit, final-publication-gate	Every report field, enrichment, coverage, gaps and hashes; the draft is labelled DRAFT_EVIDENCE_BACKED	None. A person authorizes the final

3 How a review moves through the system



Independent evidence families run at the same time and meet at explicit rendezvous points. A rendezvous waits for every expected member and classifies each one (succeeded, failed, blocked, timed out, missing). A deterministic merge checks identities, contracts and citations and keeps dissent. Evidence-qualified quorum applies evidence and independence thresholds, not a headcount. Roles talk through typed, hash-bound artifacts and an append-only claim ledger, never a shared chat. The component map is the shared scope key: it routes components to language tools, threat work, standards, native analysis and hunters.

4 Why it scales

A person reviews one component at a time. The process splits the same work along the lines the target already has and runs the pieces in parallel under fixed limits.

- **Parallel by component and language.** The eight per-language CodeQL nodes run side by side; a compiled language without a successful build is a gap, not a failure (ADR-0023). Native analysis runs per compile invocation and build unit, so one failure is one gap (ADR-0014).
- **Sharded reviewer pools.** Each review stage splits its claims across reviewer instances (default 3, three calls at once), each reviewing a different shard as a different persona. Each claim is reviewed once per stage (ADR-0021). Hunters shard the code by component into three groups, components with the most P1/P2 leads first (ADR-0018). Workbench cells run one, two or three at once by budget class.
- **Persona pools.** 75 persona and 48 role records sit in `appsec-review-process/personas/`. Attackers, defenders, verifiers and scorers come from separate lists, so the persona that argues a claim is never the one that verifies it.
- **Budget tiers.** `probe`, `standard` and `full` set each model call's limits: 200k, 800k and 1.6M input tokens; 15, 30 and 60 minutes. A standard call is capped at 2 USD; the default hunter pool therefore spends at most about 6 USD. Tiers change depth, never the evidence or permission rules.
- **Shared, bounded capacity.** Dagster pools cap concurrent work across all runs (five model slots, two container slots). Three targets ran at once on one host on 2026-09-27/28.
- **Reusable pinned tools.** Tools live in local images pinned by digest and recorded before use. The same

- image serves every target; workers never pull opportunistically.
- **Offline reference feeds.** NVD and OSV are synchronised outside the review into immutable snapshots, and Grype databases are registered the same way. A 14-day ceiling (1,209,600 s) applies; a missing or stale snapshot is a gap or a block, never a silent pass. The OSV index measured on 2026-09-29 holds 283,781 advisory records, builds in about 12 s at 195 MB, and answers lookups in under 1 ms against about 6 s from the raw archives. MITRE ATT&CK, CAPEC and the full CWE catalog will use the same feed rules when brief O merges (decision log D-05, D-11).
- **Index-first lookup.** Models query pinned inputs through read-only lookup tools instead of having whole files pasted into the prompt. On freeciv21, a 6,100-file inventory is summarised by folder, about 6K characters instead of 895K.
- **Result caches.** Accepted attempts are reused on relaunch: a pure-reuse relaunch of hello-autotools ran 46 steps in 2.4 minutes (2026-09-28). A persona result cache answers an identical model request without a new call. Re-execution after a fingerprint change is still expensive: 41 minutes to re-plan 53 units on appsec-multi-vuln. The tool-output cache and per-item memo are open (brief N).
- **Dev-mode restart** (ADR-0025). In development, a job ported to the generic item executor reruns only when the content or shape of its inputs changed; one job is ported so far. Dev results are never evidence. Not yet run live.

Throughput per host and reviews per engineer-month are **TBD: measure on the four targets**.

5 Why it can be trusted

- **Evidence first.** A tool hit, a STRIDE hypothesis or a hunter's idea is a candidate. It becomes a finding only after independent verification, and every claim cites a producer, an artifact hash and a line range that consumers re-read and re-hash.
- **The pipeline gathers; it does not judge.** Deterministic jobs collect and normalise evidence. Conclusions are carried only by enumerated values checked against each job's allowed claim classes, never parsed from prose (ADR-0013).
- **Python derives every mechanical field.** Ids, hashes, lineage, citations, CVSS scores, reachability verdicts and chain states are computed, not accepted from the model. A model reply that cites a location Python cannot resolve becomes a gap.
- **Gaps stay visible.** A tool that did not run, a build that failed or an unsupported language is a coverage gap in the report, never "no issues". `OK_WITH_GAPS` is an honest result, not a failure.
- **Separate roles.** Red team proposes, blue team refutes, an independent verifier decides, and a scorer rates. Each runs under its own registry role, which limits the claim class it may emit (ADR-0024). The blue team cannot verify; nobody promotes their own proposal.
- **Severity is earned.** Critical needs a reachability witness path; without one the cap is High. The CVSS 4.0 table was checked against FIRST's calculator on 204,976 vectors with no mismatch (2026-09-29).
- **Target content is data.** Source, documents, logs and search hits under review cannot direct the process or grant permissions.
- **Pinned and reproducible.** Tools, images, reference snapshots and prompts are pinned and hashed; each job's fingerprint covers its own code and inputs. Scans run offline.
- **Dev runs are never evidence.** A dev-mode receipt carries `evidence_grade: false`; production never reuses it, and final publication refuses any run with a dev result (ADR-0025).

6 Where people stay in the loop

The process stops at these decisions; each belongs to a person.

DECISION	WHO AND WHY
Engagement scope and permissions	A person defines the target, business goal and budget, and grants every capability beyond reading source.
Risk acceptance	The process rates risk; only an accountable owner accepts it.
Final publication	The draft stays <code>DRAFT_EVIDENCE_BACKED</code> until a person authorizes the final package at the final gate.
Personas	48 of 75 persona records were generated from the catalog and are marked unreviewed; a named owner reviews them, starting with the stage reviewers and hunters (decision log D-06).
Business context and novel design flaws	Business logic, intent and design flaws no evidence points to are outside what the process can see.
Decision-log entries	The owner accepts or overrides each controller decision (decision log, 2026-09-29).

What it does not do

- It does not replace an AppSec engineer or find every vulnerability.
- It does not exploit anything live, probe production or run authenticated dynamic tests. PoCs and fixes are static text; nothing is executed, compiled or applied.
- It does not certify compliance. OWASP, STIG/SRG and deployment output are worklists and static observations.
- It does not treat a missing scan as a pass, or an attack chain as a verified finding.
- It does not publish a final report without human authorization.

7 Readiness

Built and unit tested

All 81 jobs have a real worker in `full_review` (readiness view: 44 deterministic Python, 27 pinned containers, 6 pool coordinators, 3 persona jobs, 1 join controller). The review lanes (hunters, sharded reviewer pools and stage roles, threat workbench and privacy, dependency reachability engines, attack chains, lane 12b, CVSS and reachability in the report, dev mode) merged on 2026-09-28/29 and are tested with unit tests and stub model replies only.

Qualified

CAPABILITY	DATE	WHAT WAS SHOWN
Ten jobs: intake, Scorecard, partition discovery, build index, classify, plan, resolution, configure, native build, source SAST	per readiness view	● <code>implemented_and_qualified</code>
Binary hardening	2026-09-27	● Real ELF built in the pinned image, routed to Checksec offline, accepted

CAPABILITY	DATE	WHAT WAS SHOWN
Offline dependency path	2026-09-27	● Syft SBOM to offline Gype and OSV to SCA to reachability evidence
Go, Java and PHP source SAST	2026-09-27	● All five language adapters through the pinned container boundary
OWASP control chain	2026-09-27	● On a Freeciv-like fixture, not a target
Synthesis report	2026-09-27	● Fixture happy path; not a target assessment
CVSS 4.0 scorer	2026-09-29	● 270/270 table entries and 204,976 vectors against FIRST

Not yet run live

Under ADR-0013, hello-autotools, freeciv21 and doom3-bfg ran `full_review` concurrently on 2026-09-27/28, and appsec-multi-vuln ran separately. The furthest points were claim review on appsec-multi-vuln (61 steps accepted) and 54 steps on hello-autotools. Each breakage is logged with its fix or as open in the TODO breakage log. The open gaps:

- **Image rebuilds pending.** CodeQL, language-server, build-environment and reverse-tool images must be rebuilt and registered in WSL. Until then every `02-codeql-<lang> node` is an UNAVAILABLE gap and CodeQL reachability rows are unknown.
- **First live run** of the hunters, the threat-workbench cells, the per-stage reviewer roles and sharded pools, lanes 14 and 12b, and dev mode.
- **Windows targets.** The clang-cl route (ADR-0003) has not built a Windows target; doom3-bfg's units became coverage gaps on Linux.
- **Reference data.** No EPSS/KEV snapshot is imported yet (the report says “not assessed”). The MITRE feed (brief O) and exported-symbol entry points (brief Q) are not merged.

8 Risk and next decisions

RISK	CONTROL AND NEXT ACTION
Unproven end to end	Rerun in order (D-15): merge briefs L, O, K; rebuild images; run hello-autotools in production mode, then iterate in dev mode with <code>--explain</code> ; then appsec-multi-vuln and freeciv21.
Incomplete coverage looks clean A model overstates a finding	Coverage denominators and gaps are mandatory in every report section. Separate red, blue, verifier and scorer roles; cited evidence; verified-only findings; reachability cap on Critical.
Model cost grows with lead count	Per-call caps; first live hunter run capped at 3×2 USD (D-03); record real cost and tune shard sizes. Per-review cost: TBD: first runs .
Unreviewed personas	Hand-review the stage reviewer and hunter personas first (D-06); a failed reviewer shard fails the stage until there is cost data.
Relaunch cost slows fixes	Narrow job fingerprints to real inputs and add the tool-output cache (D-13, brief N).
Reference data goes stale	14-day ceiling; missing or stale NVD/OSV data blocks or fails the lane; stale MITRE tags are withheld as a gap (D-11).
Draft published as final	Human authorization and fail-closed final-publication checks; dev results refused.
Sample report mistaken for a run	Sample section 3C records stay labelled as sample data until real output replaces them (D-10).

Small open items from the decision log: stage 12 runtime wording says `scorer` (D-08); the repository needs

Python 3.11 or newer (D-14).

A Evidence, search and reference data

FAMILY	REPRESENTATIVE RETAINED EVIDENCE
Source	Language SAST including CodeQL per language; secrets; mobile; config and IaC findings.
Build and native	Compile database, build units, LLVM IR, Joern CPG/AST, symbols, tests, memory leads and ELF hardening.
Supply chain	Image inventory, SBOMs, licences, lifecycle/EOL signals, offline SCA matches and dependency reachability.
Context	Discovery, documents, APIs, standards snapshots, threat-model and privacy inputs.

Two search planes serve different questions. The full-text index finds literal text, configuration and documentation in frozen, hashed evidence. Joern CPG/AST and LLVM IR answer structural questions about calls, symbols and data flows. Search results locate evidence; they do not become evidence by being copied into a prompt. Cross-references link partitions, components, files, symbols, build units, dependencies, controls, threats, claims and report citations, so a reader can move from a report statement back to the exact code or tool output.

B Authority and operation

The machine-readable job graph, design-parity manifest, registry templates and contracts, and accepted run artifacts are authoritative. The generated job catalog describes the full topology and the readiness view gives per-job status. This document summarizes them for leadership; it grants no completion credit. Operators stage the engagement, launch `full_review`, and follow the operator guide for commands and stop conditions. Configuration changes belong in versioned registry templates, contracts, image pins or snapshot policy, never in target content or report edits.

C Revision history

VERSION	DATE	AUTHOR	CHANGE
0.5	2026-09-27	W. Sollers	Documented automatic plan derivation, wave dispatch and three remaining gates.
0.6	2026-09-29	W. Sollers	Reframed as an automated AppSec engineer: task-to-job map, scaling, trust, human decisions, readiness from the 2026-09-29 decision log.